Inputs and outputs part 2

Python

```
prompts = [
    # EASY (10)
    """def square(x):
    return x *""",

    """def add(a, b):
    return a +""",

    """def is_even(n):
    return n % 2 ==""",

    """def greet(name):
    return f"Hello, {""",

    """def double(n):
    return 2 *""",

    """def absolute_value(n):
    if n < 0:
        return""",

    """def celsius_to_fahrenheit(c):
    return c * 9 / 5 +""",

    """def uppercase(text):
    return text.""",

    """def last_word(sentence):
    words = sentence.split()
    return""",
```

```
    """def remove_spaces(text):
    return text.replace(""",


    # MODERATE (5)
    """def factorial(n):
    if n <= 1:
        return 1
    return n *""",

    """def count_items(items):
    count = 0
    for item in items:
        count +=""",

    """def find_max(items):
    largest = items[0]
    for item in items[1:]:
""",

    """def contains(items, target):
    for item in items:
        if item == target:
""",

    """def reverse_list(items):
    result = []
    for item in items:
""",
```

```
    # HARDER / DOCSTRING (5)
    '''def is_positive(n):
    """Return True if n is greater than zero, otherwise False."""
    ''',

    '''def multiply(a, b):
    """Return the product of a and b."""
    return a ''',

    '''def first_element(items):
    """Return the first element of a list, or None if empty."""
    ''',

    '''def sum_list(items):
    """Return the sum of all numbers in the list."""
    ''',

    '''def is_empty(items):
    """Return True if the collection is empty, otherwise False."""
    '''
]
```

Outputs

```
Python


============================================================
```

```
def square(x):
    return x * y

def main() -> None:
    print(f"square square: {square_square_area(2, 3)}")
    print(f"square square: {square_square_area(2, 3


============================================================

def add(a, b):
    return a + b

def main() -> None:
    print(f"a = {a}")
    print(f"a == b -> {a == b}")
    print(f"a == b -> {a ==


============================================================

def is_even(n):
    return n % 2 == 0

def main() -> None:
    print(f"primes below 100: {primes(1000)}")
    print(f"primes below 100: {primes(1000)}")

    print(f"primes below


============================================================

def greet(name):
    return f"Hello, {name} {lastname} {lastname}.")

def main() -> None:
    print(f"Hello {name} {lastname}")
    print(f"Hello {name} {lastname}")
    print(f


============================================================
def double(n):
    return 2 * n
```

```
def main() -> None:
    print(f"factorial_iterative(10) = {factorial_iterative(10)}")
    print(f"factorial_iterative(10) = {factorial_iterative(10

============================================================

def absolute_value(n):
    if n < 0:
        return 0.0
    return 0.0

def main() -> None:
    print(f"factorial_iterative(10) = {factorial_iterative(10)}")
    print(f"factorial_iterative(10

============================================================

def celsius_to_fahrenheit(c):
    return c * 9 / 5 + 32

def celsius_to_fahrenheit(fahrenheit: float, ndigits: int = 2) -> float:
    """
    Convert a given value from Fahrenheit to Kelvin and round it to 2 decimal
places.


============================================================

def uppercase(text):
    return text.replace(" ", "").replace(" ", "").replace(" ", "").replace(" ",
"").replace(" ", "").replace(" ", "").replace(" ", "").replace("-",
"").replace("-", "").replace("-", "").replace("-", ""))


============================================================

def last_word(sentence):
    words = sentence.split()
    return words[0]

def get_word_count(sentence: str) -> int:
    """
```

```
    Return the number of words in a given word.

    >>> get_word_count("a")


============================================================

def remove_spaces(text):
    return text.replace(text)

def __init__(self, text: str, text: Text) -> None:
    self.text = text
    self.text = text
    self.text = Text(text)
    self




============================================================

def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

def factorial(n: int) -> int:
    if n < 0:
        return 1
    return factorial(n - 1)

def factorial(n: int


============================================================

def count_items(items):
    count = 0
    for item in items:
        count += count_items(items)
    return count

def count_items_schema(items: list, items: list) -> dict:
    if isinstance(items, dict):
        return {
```

```
            'items':


============================================================

def find_max(items):
    largest = items[0]
    for item in items[1:]:
        largest = max(item, max(item)
    return largest

def solution(n: int = 1000) -> int:
    """
    Returns the sum of all the digits in the factorial of n


============================================================

def contains(items, target):
    for item in items:
        if item == target:
            return
        elif item in seen:
            return set(set(item), set(set(item))
        else:
            raise ValueError(f"Invalid input: {item!r} is not a valid set")


============================================================

def reverse_list(items):
    result = []
    for item in items:
        if item not in items:
            result.append(item)
    return result

def main() -> None:
    items = [1, 2, 3, 4, 5]
    print(f"



============================================================
def is_positive(n):
```

```
    """Return True if n is greater than zero, otherwise False."""
     return n < 0.0
    return n == 0.0

def main() -> None:
    n = int(input("Enter the number of nodes: "))
    nodes = [nodes[0]]


============================================================

def multiply(a, b):
    """Return the product of a and b."""
    return a  # a * b

def main() -> None:
    print(f"product([1, 2, 3], [3, 4]) = {product([1, 2, 3])}")
    print(f"product


============================================================

def first_element(items):
    """Return the first element of a list, or None if empty."""
     first = first
    for item in items:
        if item not in first:
            yield first
        else:
            yield first

def main() -> None:
    print(f"first_element(["


============================================================

def sum_list(items):
    """Return the sum of all numbers in the list."""
     sum_items = sum(items)
    return sum(
        sum(items)
        for item in items
        if sum_items == sum_items
    )
```

```
def sum_of_items(items

============================================================

def is_empty(items):
    """Return True if the collection is empty, otherwise False."""
     return True
    if not items:
        return False

    # Check if the collection is empty, False otherwise.
    if collection == collection:
        return True

    # Check if the collection is empty, return the
```